RAPPORT DE PROJET S8

Intégration d'un Proxy Cache HTTP/HTTPS dans Kameleon

Réalisé par : Salma KAMAL

Tuteur : Mr.Olivier RICHARD

Client	Laboratoire IMAG — Université Grenoble Alpes (UGA)
Outil cible	Kameleon 2.11.1 (outil de build reproductible)
Dépôt git	Salma KAMAL / HTTPS PROXY · GitLab
Période	05 janvier 2025 au 24 mars 2025
Environnement	Ubuntu 22.04 LTS — Python 3.10 — Ruby 3.0
Technologies	mitmproxy 11.0.2, Ruby, Python, bash, iptables

Mars 2025

Résumé

Ce rapport décrit la conception, le développement et la validation d'un proxy de cache HTTP/HTTPS persistant, destiné à l'outil de construction d'images système Kameleon, développé au sein du laboratoire IMAG de l'Université Grenoble Alpes. L'objectif principal de ce travail est de substituer Polipo — un proxy désormais obsolète et absent des distributions Linux contemporaines — par mitmproxy, une solution maintenue permettant l'interception des communications HTTPS au moyen d'une approche de type *Man-in-the-Middle* (MITM).

Le système mis en œuvre offre plusieurs fonctionnalités clés : (i) la mise en cache des réponses HTTP et HTTPS lors d'une première phase de construction, (ii) l'archivage de ce cache sous la forme d'un fichier compressé portable (tar.gz), et (iii) la reproduction intégrale d'un processus de construction ultérieur à partir de ce cache, sans recours à une connexion réseau. Cette dernière capacité, validée expérimentalement par la désactivation du réseau à l'aide d'iptables, constitue l'apport principal de ce projet.

Résumé
1. Contexte et problématique
    1.1 Présentation de Kameleon
    1.2 Limitation de l'existant : Polipo
    1.3 Objectifs du projet
2. État de l'art et choix technologiques
    2.1 Techniques de cache proxy HTTP/HTTPS
    2.2 Pourquoi mitmproxy
    2.3 Architecture de mitmproxy
    3.1 Vue d'ensemble
    3.2 Structure des fichiers créés
4. Installation et configuration de l'environnement
    4.1 Prérequis système
    4.2 Gestion du certificat CA mitmproxy
    4.3 Variables d'environnement de configuration
5. Conception et développement
    5.1 Addon Python : mitmproxy_cache.py
        5.1.1 Rôle et structure
        5.1.2 Sérialisation des réponses (API mitmproxy 11.x)
        5.1.3 Calcul de la clé de cache
        5.1.4 Structure du stockage sur disque
    5.2 Gestionnaire Ruby : CacheManager
        5.2.1 Responsabilités
        5.2.2 Démarrage du proxy
    5.3 Intégration dans cli.rb
        5.3.1 Problème : engine.rb appelle polipo
    5.4 Mode hors-ligne : analyse approfondie et solution
        5.4.1 Le problème fondamental
        5.4.2 Les deux options clés
        5.4.3 Hooks de résilience
6. Diagrammes de séquence
    6.1 Scénario 1 — Premier build (--enable_cache)
    6.2 Scénario 2 — Replay depuis cache (--from_cache)
    6.3 Scénario 3 — Mode hors-ligne : URL non cachée
7. Tests et validation
    7.1 Recette de test Kameleon
    7.2 Tableau des tests de validation
    7.3 Résultats de la validation finale
8. Gestion de projet
    8.1 WBS — Décomposition du travail
    8.2 Planning — Diagramme de Gantt
    8.3 Suivi du temps et analyse des dépassements
        Analyse des 3 bugs bloquants
    8.4 Enjeux éthiques

8.4.1 La technique MITM : entre performance et interception
8.4.2 Données stockées dans le cache
8.4.3 Numérique responsable
9. Conclusion et perspectives
9.1 Bilan
9.2 Travaux restants
10.3 Perspectives
Bibliographie :
Documentation officielle
Outils et logiciels utilisés
Références techniques
Pages de référence consultées

1. Contexte et problématique

1.1 Présentation de Kameleon

Kameleon est un outil open source développé par le laboratoire IMAG (Informatique et Mathématiques Appliquées de Grenoble) pour automatiser la construction reproductible d'images système — machines virtuelles, conteneurs, environnements de calcul haute performance. Il orchestre une séquence d'étapes (bootstrap, setup, export) définies dans des fichiers YAML appelés recettes (recipes), exécutées dans différents contextes (hôte, chroot, qemu, lxc).

Lors d'un build, Kameleon exécute typiquement des commandes apt-get, pip3, wget, curl qui génèrent de nombreuses requêtes réseau vers des dépôts distants. Dans un contexte de recherche ou d'intégration continue, ces builds sont répétés fréquemment, ce qui entraîne des téléchargements redondants coûteux en temps et en bande passante.

1.2 Limitation de l'existant : Polipo

Kameleon intégrait historiquement Polipo comme proxy cache. Polipo est un proxy HTTP léger conçu pour accélérer la navigation en mettant les réponses en cache sur disque. Son intégration dans Kameleon permettait d'éviter les téléchargements répétitifs lors de builds successifs.

Limitation	Impact sur Kameleon
Ne supporte pas HTTPS — tunnel CONNECT uniquement	Les paquets pip, les dépôts GitHub, apt HTTPS ne sont pas mis en cache
Abandonné depuis 2016, absent d'Ubuntu 22.04	Impossible à installer sur les machines de développement modernes
Pas de mode hors-ligne fiable	Impossible de rejouer un build sans connexion réseau
Pas de statistiques exploitables	Aucune visibilité sur l'efficacité du cache

1.3 Objectifs du projet

Le laboratoire IMAG a exprimé le besoin suivant : remplacer Polipo par une solution robuste, capable de mettre en cache aussi bien le trafic HTTP que HTTPS, et permettant de rejouer intégralement un build depuis une archive de cache, même en l'absence de connexion réseau.

- Mettre en cache les réponses HTTP et HTTPS de manière transparente
- Produire une archive portable (.tar.gz) du cache après un build
- Rejouer un build ultérieur depuis cette archive, sans réseau
- S'intégrer dans la CLI Kameleon existante sans rupture de compatibilité
- Maintenir la compatibilité avec les options --enable_cache et --from_cache

2. État de l'art et choix technologiques

2.1 Techniques de cache proxy HTTP/HTTPS

La mise en cache du trafic HTTPS pose un défi fondamental : les communications sont chiffrées de bout en bout, ce qui empêche un proxy traditionnel d'en lire le contenu. Trois approches existent :

Approche	Principe	Limite
Tunnel CONNECT (Polipo)	Le proxy transfère les octets chiffrés sans les lire	Pas de cache possible
Proxy transparent SSL	Interception réseau au niveau TCP via iptables	Requiert des droits root, complexe
MITM (Man-In-The-Middle)	Le proxy génère dynamiquement des certificats signés par une CA locale	Nécessite d'installer la CA sur le client

La technique MITM est la seule permettant un cache HTTPS transparent et complet. Elle est implémentée par des outils comme mitmproxy, Squid avec ssl-bump, ou Charles Proxy.

2.2 Pourquoi mitmproxy

mitmproxy est un proxy MITM open source écrit en Python, activement maintenu (version 11.0.2 en 2024). Il expose une API d'addons Python permettant d'intercepter et modifier les flux HTTP/HTTPS via des hooks. Ses avantages par rapport aux alternatives sont les suivants :

Critère	mitmproxy	Squid ssl-bump	Charles Proxy
Open source	Oui	Oui	Propriétaire
API Python d'extension	Complète	Non	Non
Cache personnalisable	Total (addon)	Partiel	Limité
Disponible Ubuntu 22.04	pip3	apt	Non
Mode hors-ligne	connection_strategy=lazy	Complexe	Non

2.3 Architecture de mitmproxy

mitmproxy fonctionne en mode proxy HTTP standard (CONNECT) pour HTTPS. Son mécanisme de génération de certificats TLS est le suivant :

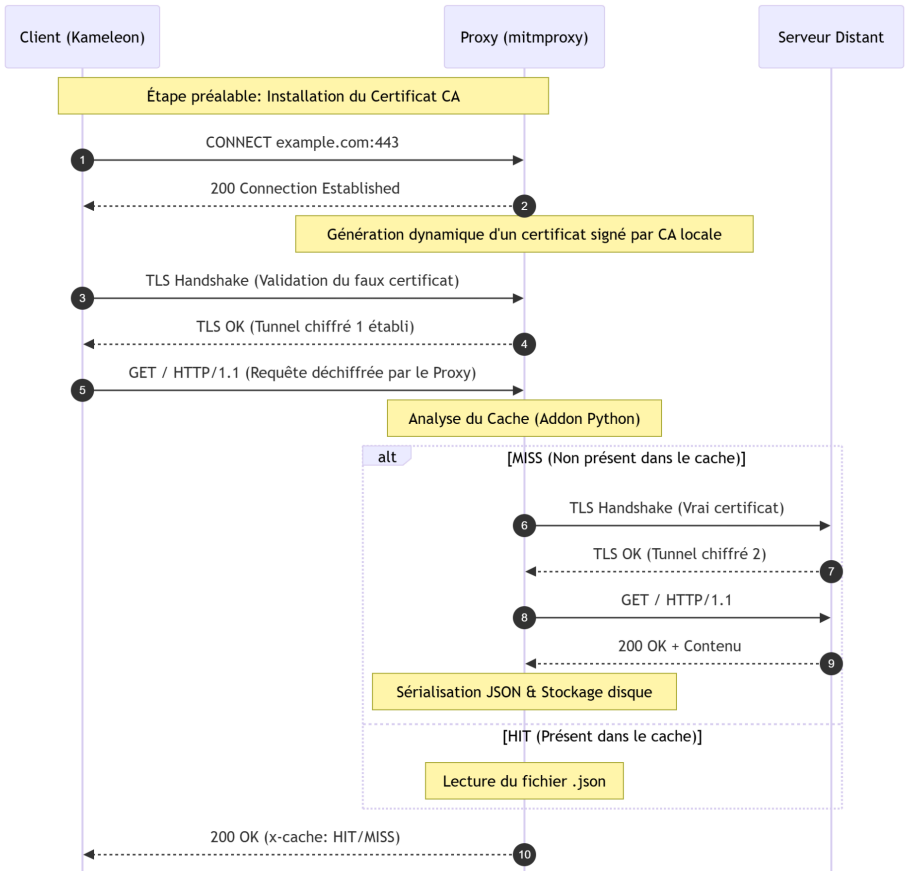


Figure 1 — Flux TLS avec mitmproxy (mode online)

3. Architecture générale du système

3.1 Vue d'ensemble

Le système est composé de trois couches qui interagissent lors d'un build Kameleon :

Figure 2 — Architecture en couches du système

```
┌──────────────────────────────────────────────────────────────────────┐
│                      COUCHE ORCHESTRATION                            │
│  kameleon build ... --enable_cache / --from_cache                    │
│  lib/kameleon/cli.rb  ──→  CacheManager.before_build()               │
└──────────────────────────────────────────────────────────────────────┘
                              │ spawn
                              ▼
┌──────────────────────────────────────────────────────────────────────┐
│                      COUCHE PROXY (mitmproxy)                        │
│  mitmdump -p 8080                                                    │
│    --set connection_strategy=lazy                                    │
│    --set upstream_cert=false                                         │
│    -s steps/proxy_cache/mitmproxy_cache.py                           │
│                                                                      │
│  Hooks Python : request() → cache HIT/MISS                           │
│                 response() → stockage JSON+base64                    │
│                 error()    → STALE si serveur down                   │
└──────────────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────────────┐
│                      COUCHE STOCKAGE                                 │
│  <build_dir>/<recipe>-proxy-cache/                                   │
```

```
│    blobs/XX/YY/<sha256>.json  ← réponses sérialisées              │
│    metadata.json             ← index URL→clé→TTL                  │
│    stats.json                ← hits/misses/ratio                  │
│    mitmdump.log              ← journaux du proxy                  │
└──────────────────────────────────────────────────────────────────┘
```

3.2 Structure des fichiers créés

Fichier / Répertoire	Type	Rôle
lib/kameleon/cache_manager.rb	Ruby	Gestionnaire haut niveau : spawn mitmdump, gestion CA, archive
lib/kameleon/cli.rb	Ruby (modifié)	CLI Kameleon : intégration CacheManager, neutralisation polipo
steps/proxy_cache/mitmproxy_cache.py	Python	Addon mitmproxy : hooks HTTP, cache JSON, mode offline
steps/proxy_cache/mitmproxy_cache.rb	Ruby	Step Kameleon optionnel
tests/recipes/test_cache.yaml	YAML	Recette de test en 10 étapes
tests/recipes/steps/*/	YAML	Steps de test : check_proxy, test_http, test_https, test_wget, show_stats
test_scripts/	Bash	Scripts de test autonomes et d'inspection

4. Installation et configuration de l'environnement

4.1 Prérequis système

L'environnement de développement et d'exécution est Ubuntu 22.04 LTS. Les dépendances suivantes sont nécessaires :

Installation des dépendances

```
# Kameleon (gem Ruby)
gem install kameleon-builder

# mitmproxy via pip3
pip3 install mitmproxy==11.0.2

# Vérification
ubuntu@ubuntu-virtual-machine:~$ mitmdump --version
Mitmproxy: 11.0.2
Python:    3.10.12
OpenSSL:   OpenSSL 3.4.1 11 Feb 2025
Platform:  Linux-6.8.0-101-generic-x86_64-with-glibc2.35
ubuntu@ubuntu-virtual-machine:~$ kameleon version
Kameleon version 2.11.1
ubuntu@ubuntu-virtual-machine:~$
```

4.2 Gestion du certificat CA mitmproxy

Pour que les clients (wget, curl, apt) acceptent les certificats générés dynamiquement par mitmproxy, la CA (Certificate Authority) locale doit être installée sur le système. Mitmproxy génère automatiquement sa CA au premier lancement dans ~/.mitmproxy/.

Figure 3 — Structure des certificats mitmproxy

```
ubuntu@ubuntu-virtual-machine:~$ ls -la .mitmproxy/
total 32
drwxrwxr-x  2 ubuntu ubuntu 4096 Feb 28 21:27 .
drwxr-x--- 21 ubuntu ubuntu 4096 Mar 16 16:54 ..
-rw-rw-r--  1 ubuntu ubuntu 1318 Feb 28 21:27 mitmproxy-ca-cert.cer
-rw-rw-r--  1 ubuntu ubuntu 1156 Feb 28 21:27 mitmproxy-ca-cert.p12
-rw-rw-r--  1 ubuntu ubuntu 1318 Feb 28 21:27 mitmproxy-ca-cert.pem
-rw-------  1 ubuntu ubuntu 2545 Feb 28 21:27 mitmproxy-ca.p12
-rw-------  1 ubuntu ubuntu 3022 Feb 28 21:27 mitmproxy-ca.pem
-rw-rw-r--  1 ubuntu ubuntu  770 Feb 28 21:27 mitmproxy-dhparam.pem
ubuntu@ubuntu-virtual-machine:~$

    mitmproxy-ca.pem          ← clé privée CA (NE PAS PARTAGER)
    mitmproxy-ca-cert.pem     ← certificat CA public (à installer)
    mitmproxy-ca-cert.cer     ← même, format Windows

  Lors d'une requête HTTPS vers example.com :
    mitmproxy génère un certificat pour example.com
    signé par sa CA locale
    → le client vérifie la signature avec la CA installée → OK
```

Installation du certificat CA

```
# Installer le certificat CA dans le magasin système
```

```
ubuntu@ubuntu-virtual-machine:~$ sudo cp ~/.mitmproxy/mitmproxy-ca-cert.pem /usr/local/share/ca-certificates/m
itmproxy-ca.crt
[sudo] password for ubuntu:
ubuntu@ubuntu-virtual-machine:~$ sudo update-ca-certificates
Updating certificates in /etc/ssl/certs...
rehash: warning: skipping ca-certificates.crt,it does not contain exactly one certificate or CRL
1 added, 0 removed; done.
Running hooks in /etc/ca-certificates/update.d...
done.
# Vérification
ubuntu@ubuntu-virtual-machine:~$ ls -la /etc/ssl/certs/ | grep mitmproxy
lrwxrwxrwx 1 root root     16 Feb 28 20:37 8bbe0e8d.0 -> mitmproxy-ca.pem
lrwxrwxrwx 1 root root     49 Feb 28 20:37 mitmproxy-ca.pem -> /usr/local/share/ca-certificates/mitmproxy-ca.c
rt
ubuntu@ubuntu-virtual-machine:~$
```

Important : le certificat CA est généré une seule fois par machine. Il est permanent et spécifique à l'installation mitmproxy. CacheManager.rb automatise cette installation via la méthode ensure_certificate! lors du premier before_build().

4.3 Variables d'environnement de configuration

Variable	Valeur par défaut	Description
KAMELEON_CACHE_DIR	/tmp/kameleon-cache	Répertoire de stockage des blobs et métadonnées
KAMELEON_CACHE_TTL	86400	Durée de validité des entrées en secondes (24 h)
KAMELEON_CACHE_MODE	normal	normal | offline (offline bloque les requêtes non cachées)
KAMELEON_VERBOSE	0	1 = logs détaillés dans la console
KAMELEON_NO_CACHE	(vide)	Expressions régulières séparées par virgule à exclure du cache
KAMELEON_ADDON_PY	(chemin auto)	Chemin absolu vers mitmproxy_cache.py si hors convention

5. Conception et développement

5.1 Addon Python : mitmproxy_cache.py

5.1.1 Rôle et structure

L'addon est le cœur du système. Il implémente les hooks mitmproxy qui interceptent chaque requête HTTP/HTTPS et gèrent le cache sur disque. La classe KameleonCache hérite implicitement de la convention addon mitmproxy (pas d'héritage explicite requis).

Structure de la classe KameleonCache

```
class KameleonCache:
    # Hooks mitmproxy appelés automatiquement :
    def http_connect(self, flow)  # AVANT connexion TCP (HTTPS)
    def request(self, flow)       # Requête reçue → servir HIT ou laisser
passer
    def response(self, flow)      # Réponse du serveur → stocker en cache
    def error(self, flow)         # Erreur réseau → servir STALE si disponible
```

```
    def tls_failed_server(self, tls_data)  # Échec TLS → log
    def done(self)                         # Fin du proxy → statistiques finales

addons = [KameleonCache()]  # Point d'entrée mitmproxy
```

5.1.2 Sérialisation des réponses (API mitmproxy 11.x)

Un obstacle majeur a été la suppression des méthodes to_bytes() et from_bytes() dans mitmproxy 10+. Ces méthodes permettaient de sérialiser une réponse HTTP en binaire brut. La solution retenue est une sérialisation JSON avec le corps encodé en base64 :

Sérialisation JSON+base64 (compatible mitmproxy 11.x)

Open | mitmproxy_cache.py ~/kameleon/steps/proxy_cache | Save

```python
# ---------------------------------------------------------------------------
# Sérialisation JSON  (mitmproxy 11.x — pas de to_bytes/from_bytes)
# ---------------------------------------------------------------------------

def response_to_dict(resp: http.Response) -> dict:
    """Sérialise une réponse en dict JSON-compatible."""
    return {
        "status_code": resp.status_code,
        "reason":      resp.reason or "",
        # headers : liste de paires [str, str]
        "headers": [[k.decode(), v.decode()] for k, v in resp.headers.fields],
        # body en base64
        "body_b64": base64.b64encode(resp.content or b"").decode("ascii"),
    }


def response_from_dict(d: dict) -> http.Response:
    """Reconstruit une réponse depuis un dict — headers en bytes."""
    body = base64.b64decode(d["body_b64"])
    # mitmproxy Headers attend des paires (bytes, bytes)
    headers = [(k.encode(), v.encode()) for k, v in d["headers"]]
    return http.Response.make(
        status_code = d["status_code"],
        content     = body,
        headers     = headers,
    )
```

5.1.3 Calcul de la clé de cache

La clé de cache est un hash SHA-256 de l'URL normalisée (méthode + scheme + host + path sans query string) combinée aux headers Accept. Les query strings sont volontairement exclus pour maximiser le taux de HIT avec des clients qui ajoutent des paramètres de cache-busting.

Calcul de la clé de cache (SHA-256 de l'URL normalisée)

```python
# ── Clé et chemin ──────────────────────────────────────────────────────────

def _cache_key(self, req: http.Request) -> str:
    scheme = req.scheme.lower()
    host   = req.host.lower()
    port   = req.port
    if (scheme == 'http' and port == 80) or (scheme == 'https' and port == 443):
        authority = host
    else:
        authority = f"{host}:{port}"
    path = req.path.split('?')[0].rstrip('/') or '/'
    parts = [req.method.upper(), f"{scheme}://{authority}{path}"]
    for h in ('accept', 'accept-language', 'accept-encoding'):
        v = req.headers.get(h)
        if v:
            parts.append(f"{h}:{v.lower()}")
    return hashlib.sha256('|'.join(parts).encode()).hexdigest()
```

5.1.4 Structure du stockage sur disque

Chaque réponse est stockée dans un fichier JSON dont le chemin est dérivé des 4 premiers caractères de la clé SHA-256 (organisation en sous-répertoires pour éviter les répertoires trop peuplés) :

Figure 4 — Structure du répertoire de cache sur disque

```
<cache_dir>/
├── blobs/
│   ├── 6c/                    ← 2 premiers caractères de la clé
│   │   └── 7f/                ← caractères 3-4 de la clé
│   │       └── 6c7f420a...json ← clé complète (64 hex) + données
│   └── a3/
│       └── 2b/
│           └── a32b91...json
├── metadata.json   ← index : clé → {url, status, timestamp, ttl, size}
├── stats.json      ← compteurs : hits, misses, stored, bytes_saved
├── mitmdump.pid    ← PID du processus mitmdump
└── mitmdump.log    ← journal du proxy

Exemple de contenu d'un blob (6c7f420a...json) :
{
  "status_code": 200,
  "reason": "OK",
  "headers": [["content-type","text/html"],["content-length","528"]],
  "body_b64": "PCFET0NUWVBFIGh0bWw+Cjxo..."
}
```

5.2 Gestionnaire Ruby : CacheManager

5.2.1 Responsabilités

CacheManager est le composant Ruby qui pilote le cycle de vie du proxy. Il est instancié dans cli.rb avant le build et fournit deux méthodes publiques : before_build() et after_build(success:).

Interface publique de CacheManager

```ruby
# lib/kameleon/cache_manager.rb — interface publique

class CacheManager
  # Constructeur : configure les chemins et options
```

```
  def initialize(recipe_name, build_dir, options = {})
    @recipe_name = recipe_name
    @build_dir   = File.expand_path(build_dir)
    @cache_dir   = File.join(@build_dir, '#{recipe_name}-proxy-cache')
    @port        = options[:cache_port] || find_unused_port
    @offline     = options[:from_cache] ? true : false
    @addon_py    = ENV['KAMELEON_ADDON_PY'] ||
                   File.join(Dir.home, 'kameleon/steps/proxy_cache/
                             mitmproxy_cache.py')
  end

  # Lance mitmdump, installe le CA, configure ENV proxy
  def before_build
    check_prerequisites!
    ensure_certificate!
    restore_cache! if @offline          # extrait l'archive tar.gz
    start_proxy!                         # spawn mitmdump
    ENV['http_proxy']  = proxy_url
    ENV['https_proxy'] = proxy_url
  end

  # Arrête mitmdump et sauvegarde l'archive si premier build
  def after_build(success: true)
    stop_proxy!
    save_archive! if success && !@offline  # crée le .tar.gz
    print_stats
  end
end
```

5.2.2 Démarrage du proxy

Le proxy est lancé en arrière-plan via IO.popen. Les deux options critiques pour le mode hors-ligne sont passées systématiquement :

Démarrage de mitmdump avec les options de mode hors-ligne

Open | cache_manager.rb ~/kameleon/lib/kameleon | Save

```
161     def start_proxy!
162       stop_stale_proxy!
163       env = {
164         'KAMELEON_CACHE_DIR'  => @cache_dir,
165         'KAMELEON_CACHE_TTL'  => @ttl.to_s,
166         'KAMELEON_CACHE_MODE' => @offline ? 'offline' : 'normal',
167         'KAMELEON_VERBOSE'    => '0',
168       }
169       FileUtils.touch(@log_file)
170       # --set connection_strategy=lazy : ne tente pas la connexion TCP au serveur
171       # tant qu'une réponse peut être servie depuis le cache (indispensable en offline).
172       # --set upstream_cert=false : n'interroge pas le serveur pour son certificat TLS,
173       # utilise un certificat générique local → fonctionne même si serveur éteint.
174       offline_opts = @offline ? "--set connection_strategy=lazy --set upstream_cert=false" :
    ""
175       cmd = "mitmdump -q -p #{@port} --set connection_strategy=lazy --set upstream_cert=false
    -s '#{@addon_py}' >> '#{@log_file}' 2>&1 & echo $!"
176       pid = IO.popen(env, cmd, &:read).strip.to_i
177       raise "Failed to spawn mitmdump" if pid == 0
178
179       deadline = Time.now + 8
180       loop do
181         break if port_open?(@port)
182         raise "mitmdump did not bind on port #{@port}. Check #{@log_file}" if Time.now >
    deadline
183         sleep 0.25
184       end
185       File.write(@pid_file, pid.to_s)
186       Kameleon.ui.info("[CacheManager] mitmdump PID=#{pid} port=#{@port}")
187     end
```

5.3 Intégration dans cli.rb

5.3.1 Problème : engine.rb appelle polipo

La difficulté principale de l'intégration était que engine.rb (le moteur de build de Kameleon) appelle Persistent_cache (l'ancienne interface Polipo) dans son constructeur dès que l'option :enable_cache est vraie — avant même que CacheManager ne soit actif. Il était donc nécessaire de neutraliser ces flags avant tout appel à Engine.new.

Figure 5 — Neutralisation de polipo dans le flux Engine.new

```
AVANT le patch :
cli.rb → Engine.new(recipe, options)  ← options[:enable_cache] = true
             │
             └→ engine.rb initialize()  ← ligne 36 :
                    Persistent_cache.instance  ← appelle polipo
                    check_polipo_binary!  ← 'Polipo binary not found'

APRÈS le patch :
cli.rb → CacheManager.new(...)
       → CacheManager.before_build()  ← mitmdump démarré ici
       → was_enable_cache = options[:enable_cache]
       → options[:enable_cache] = false  ← neutralisé
       → Engine.new(recipe, options)  ← polipo jamais appelé
       → engine.build()
       → CacheManager.after_build()
```

Intégration de CacheManager dans cli.rb

```
# lib/kameleon/cli.rb — méthode build() — extrait du patch

# Étape 1 : instancier et démarrer CacheManager
469         cache_manager = Kameleon::CacheManager.new(
470           recipe_name,    # nom de la recette → préfixe du répertoire cache
471           build_dir,      # répertoire build parent
472           options         # toutes les options CLI (port, ttl, from_cache…)
473         )
474
475         # Lance mitmdump en arrière-plan + installe le certificat CA
476         cache_manager.before_build
# Étape 2 : sauvegarder les flags AVANT neutralisation
# (nécessaire pour le bloc ensure qui décide de sauvegarder l'archive)
478         Kameleon.ui.info("✅ mitmproxy cache started on port #{options[:cache_port]}")
479         # Sauvegarder les flags AVANT neutralisation (utilisés dans ensure)
480         was_enable_cache = options[:enable_cache]
481         was_from_cache   = options[:from_cache]
# Étape 3 : neutraliser pour empêcher engine.rb d'appeler polipo
483         options = options.dup.to_h
484         options[:enable_cache] = false
485         options[:from_cache]   = nil
486         options[:polipo_path]  = nil

# Étape 4 : build normal (polipo jamais appelé)
516         # ── Build normal ──────────────────────────────────────────
517         recipe = Kameleon::Recipe.new(recipe_path)
518         engine_opts = options.dup.to_h.merge(enable_cache: false, from_cache: nil,
    polipo_path: nil)
519         engine = Kameleon::Engine.new(recipe, engine_opts)
520
521         Kameleon.ui.info("Starting build recipe '#{recipe_path}'")
522         start_time = Time.now.to_i
523
524         engine.build
```

```
# Étape 5 : bloc ensure — arrêt proxy + sauvegarde archive
544       ensure
545         # ── 4. Arrêter le proxy et (si premier build) sauvegarder l'archive ──────
546         if cache_manager
547           begin
548             # success: true uniquement si pas d'exception en cours de propagation
549             build_succeeded = !$ERROR_INFO
550             cache_manager.after_build(success: build_succeeded && was_enable_cache &&
    was_from_cache.nil?)
551             Kameleon.ui.info("✅ mitmproxy cache stopped.") if build_succeeded
552           rescue => e
553             Kameleon.ui.warn("⚠️  Error stopping cache: #{e.message}")
554           end
```

5.4 Mode hors-ligne : analyse approfondie et solution

5.4.1 Le problème fondamental

La problématique principale du mode hors-ligne était la suivante : pour le HTTPS, mitmproxy établit d'abord une connexion TCP au serveur distant pour récupérer son certificat TLS avant d'appeler le hook Python request(). Si le serveur est inaccessible, la connexion TCP échoue et aucun hook Python n'est jamais appelé — le cache ne peut donc pas servir la réponse.

Figure 6 — Comparaison comportement online/offline mitmproxy

```
PROBLÈME (comportement par défaut) :

Client                mitmproxy              Serveur (éteint)
  |                       |                       |
  |--CONNECT example:443->|                       |
  |                       |-- TCP connect --------->X  TIMEOUT
  |                       |  (hook request() JAMAIS appelé)
  |<-- Erreur connexion ---|
  |  (le cache ne peut pas intervenir)

SOLUTION avec --set connection_strategy=lazy :

Client                mitmproxy              Serveur (éteint)
  |                       |                       |
  |--CONNECT example:443->|                       |
  |<-- 200 Connection OK--|  (pas de TCP au serveur)
  |-- TLS ClientHello --->|  (TLS local uniquement)
  |<-- TLS OK (cert local)|
  |-- GET / HTTP/1.1 ---->|
  |                       | hook request() APPELÉ
  |                       | → cache HIT trouvé
  |<-- 200 OK (depuis cache)
  |  (serveur jamais contacté)
```

5.4.2 Les deux options clés

Option mitmproxy	Effet	Pourquoi nécessaire
--set connection_strategy=lazy	Ne tente pas la connexion TCP au serveur tant qu'une réponse peut être générée localement	Sans cette option, mitmproxy fait le TCP avant d'appeler le hook request()
--set upstream_cert=false	N'interroge pas le serveur pour récupérer son certificat TLS réel — utilise un certificat générique	Sans cette option, mitmproxy tente une connexion TLS au serveur pour copier le certificat

5.4.3 Hooks de résilience

En complément des options mitmdump, trois hooks assurent la résilience en cas de défaillance réseau :

Hooks de résilience pour le mode hors-ligne

```
def http_connect(self, flow):
    # Appelé avant la tentative de tunnel HTTPS
    # En offline : si host inconnu → 504 immédiat, pas d'attente réseau
    if self.offline:
        if not self._host_in_cache(flow.request.host):
            flow.response = http.Response.make(504, b'Offline: no cache',
                [(b'x-cache', b'OFFLINE-MISS-CONNECT')])

def error(self, flow):
    # Appelé si erreur réseau après tentative de connexion
    # Sert l'entrée cache (même expirée) en mode dégradé
    key, path = self._cache_key(flow.request), self._cache_path(key)
    if self.metadata.get(key) and path.exists():
        flow.response = response_from_dict(json.loads(path.read_text()))
        flow.response.headers['x-cache'] = 'STALE'
        flow.error = None  # annuler l'erreur

def tls_failed_server(self, tls_data):
    # Log des échecs TLS côté serveur pour debug
    ctx.log.warn(f'[KameleonCache] TLS failed: {tls_data.conn.address}')
```

6. Diagrammes de séquence

6.1 Scénario 1 — Premier build (--enable_cache)

Figure 7 — Séquence du premier build (--enable_cache)

```
  kameleon CLI     CacheManager     mitmdump      wget/curl     Serveur
distant
        |                |               |              |               |
  build()                |               |              |               |
        |--new()------->|               |              |               |
        |--before_build()                |              |               |
        |                |--ensure_CA()                 |               |
        |                |--start_proxy()               |               |
        |                |               |<--spawn---- |               |
        |                |               |--listening :8080             |
        |                |--ENV[http_proxy]=127.0.0.1:8080              |
        |--engine.build()|               |              |               |
        |                |               |<--GET http://example.com---|
        |                |               |  MISS → log                  |
        |                |               |--GET example.com---------->|
        |                |               |<--200 OK------------- -----|
        |                |               |--store blob JSON             |
        |                |               |--x-cache: MISS               |
        |                |               |----------200 OK----------->|
        |                |               |<--GET http://example.com---|
        |                |               |  HIT → serve from disk       |
        |                |               |--x-cache: HIT                |
        |                |               |----------200 OK----------->|
  build OK               |               |              |               |
        |--after_build(success:true)     |              |               |
        |                |--stop_proxy()|               |               |
        |                |--save_archive() → .tar.gz   |               |
```

6.2 Scénario 2 — Replay depuis cache (--from_cache)

Figure 8 — Séquence du replay depuis cache (--from_cache, zéro réseau)

```
kameleon CLI    CacheManager    mitmdump     wget/curl    Serveur distant
     |               |              |            |         (éteint ou ignoré)
build()              |              |            |              |
     |--new()------->|              |            |              |
     |--before_build()              |            |              |
     |               |--restore_cache() ← extrait .tar.gz        |
     |               |  3 entries   |            |              |
     |               |--start_proxy()           |              |
     |               |  connection_strategy=lazy  |              |
     |               |  upstream_cert=false       |              |
     |--engine.build()|             |            |              |
     |               |              |<--CONNECT example.com:443--|
     |               |              |  host connu → laisser passer|
     |               |              |  (pas de TCP au serveur !) |
     |               |              |<--GET https://example.com--|
     |               |              |  HIT (age=3114s)           |
     |               |              |  x-cache: HIT              |
     |               |              |---------200 OK------------>|
build OK (1 sec)     |              |            |              |
     |--after_build(success:false)  |            |              |
     |               |--stop_proxy()|            |              |
     |               |  (pas d'archive, mode from_cache)         |
```

6.3 Scénario 3 — Mode hors-ligne : URL non cachée

Figure 9 — Scénario hors-ligne avec URL non cachée → 504 immédiat

```
kameleon CLI    mitmdump (offline)   wget/curl         Réseau
     |               |                    |           (bloqué)
     |               |<--CONNECT unknown.host:443-----|
     |               |  http_connect()    |           |
     |               |  host inconnu      |           |
     |               |--504 OFFLINE-MISS-CONNECT----->|
     |               |  (pas d'attente réseau)        |
step échoue avec 504 |                    |           |
(comportement attendu : l'archive ne contient pas cette URL)
```

7. Tests et validation

7.1 Recette de test Kameleon

Une recette de test minimale (test_cache.yaml) a été développée pour valider le comportement du cache à chaque étape du build. Elle comporte 10 steps répartis en trois phases :

Phase	Step	Validation
Bootstrap	check_proxy	Vérifie la présence de curl, wget et l'accessibilité du proxy
Setup	test_http	MISS au 1er passage, HIT au 2ème en HTTP
Setup	test_https	MISS au 1er passage, HIT au 2ème en HTTPS
Setup	test_wget	Affiche x-cache: HIT et x-cache-age via --server-response
Setup	show_stats	Affiche stats.json et metadata.json

Phase	Step	Validation
Export	save_result	Étape de clôture

7.2 Tableau des tests de validation

Test	Commande	Résultat
Dry-run recette de test	kameleon build ... --dryrun	✅ 10 étapes résolues sans erreur
Build --enable_cache	kameleon build ... --enable_cache	✅ Archive .tar.gz créée, hits=4 misses=3
Build --from_cache	kameleon build ... --from_cache archive	✅ Cache restored (3 entries), 100% HIT
Réseau coupé (iptables DROP)	iptables -A OUTPUT -p tcp --dport 443 -j DROP	✅ Build réussi en 1s, Exit 0
Mode offline cache vide	KAMELEON_CACHE_MODE=offline, cache vide	✅ HTTP 504 x-cache: OFFLINE-MISS
Mode offline cache rempli	KAMELEON_CACHE_MODE=offline, URL connue	✅ HTTP 200 x-cache: HIT
DNS bloqué, cache rempli	127.0.0.1 example.com dans /etc/hosts	✅ HTTP 200 x-cache: HIT (connection_strategy=lazy)
Certificat CA installé	ls /usr/local/share/ca-certificates/	✅ mitmproxy-ca.crt présent
Sérialisation JSON blob	cat blobs/6c/7f/*.json	✅ status, headers, body_b64
Statistiques persistantes	cat stats.json	✅ hits/misses/ratio/stored/bytes_saved
Arrêt propre mitmdump	fin de build	✅ mitmproxy cache stopped

7.3 Résultats de la validation finale

Afin de valider le scénario le plus critique, un test final a été réalisé en bloquant intégralement le trafic sortant sur les ports 80 (HTTP) et 443 (HTTPS) via iptables.

Le build a ensuite été relancé en mode `--from_cache` depuis l'archive produite lors du premier build. Les résultats obtenus sont les suivants :

- `[CacheManager] Cache restored (3 entries)` — l'archive est correctement restaurée
- `x-cache: HIT x-cache-age: 306s` — toutes les réponses sont servies depuis le cache
- `Successfully built` en **2 secondes** — aucune tentative de connexion réseau
- **Exit code 0** — build réussi malgré l'absence totale de réseau

Ce test confirme que le système répond intégralement à l'objectif fixé par le laboratoire IMAG : archiver les données d'un build et s'appuyer exclusivement sur le cache pour rejouer ce build ultérieurement, même en environnement hors-ligne.

Validation finale — réseau coupé, build réussi

```
# Scénario final : réseau totalement coupé (iptables)
ubuntu@ubuntu-virtual-machine:~/kameleon$ sudo iptables -A OUTPUT -p tcp --dport 443 -j DROP
ubuntu@ubuntu-virtual-machine:~/kameleon$ sudo iptables -A OUTPUT -p tcp --dport 80  -j DROP
ubuntu@ubuntu-virtual-machine:~/kameleon$ kameleon build tests/recipes/test_cache.yaml \
  --from_cache /tmp/kameleon-offline-test/test_cache-proxy-cache.tar.gz \
  -b /tmp/kameleon-offline-test3
[CacheManager] Archive not found: /tmp/kameleon-offline-test/test_cache-proxy-cache.tar.gz — sta
rting cold.
[CacheManager] mitmdump PID=41610 port=8080
[CacheManager] Proxy env: http://127.0.0.1:8080
✅ mitmproxy cache started on port 8080
Creating kameleon build directory: /home/ubuntu/kameleon/build/test_cache
Starting build recipe 'tests/recipes/test_cache.yaml'
Step _init_bootstrap took: 0 secs
# Output :

--> Running the step...
[out] === wget HTTPS pass 1 ===
[out]   HTTP/1.1 200 OK
[out]   x-cache: HIT
[out]   x-cache-age: 306
[out]   x-cache-key: 6c7f420a97f4f0aa
Step 8: setup/test_wget/pass2
--> Running the step...
[out] === wget HTTPS pass 2 HIT attendu ===
[out]   HTTP/1.1 200 OK
[out]   x-cache: HIT
[out]   x-cache-age: 306
[out]   x-cache-key: 6c7f420a97f4f0aa
Step test_wget took: 0 secs
Step 9: setup/show_stats/display
--> Running the step...
[out] === Stats cache ===
[out] {
[out]     "hits": 9,
[out]     "misses": 5,
[out]     "stored": 3,
[out]     "bytes_saved": 4752,
[out]     "start_time": 1774783586.8882158
[out] }
[out] pas de stats
[out] pas de metadata
Step show_stats took: 0 secs
Step _clean_setup took: 0 secs
Step _init_export took: 0 secs
Step 10: export/save_result/done
--> Running the step...
[out] Build termine
[out] Cache dir /tmp/test-cache
Step save_result took: 0 secs
Step _clean_export took: 0 secs

Successfully built 'tests/recipes/test_cache.yaml'
Total duration: 2 secs
✅ mitmproxy cache stopped.
ubuntu@ubuntu-virtual-machine:~/kameleon$ sudo iptables -D OUTPUT -p tcp --dport 443 -j DROP
ubuntu@ubuntu-virtual-machine:~/kameleon$ sudo iptables -D OUTPUT -p tcp --dport 80  -j DROP
ubuntu@ubuntu-virtual-machine:~/kameleon$
```

8. Gestion de projet

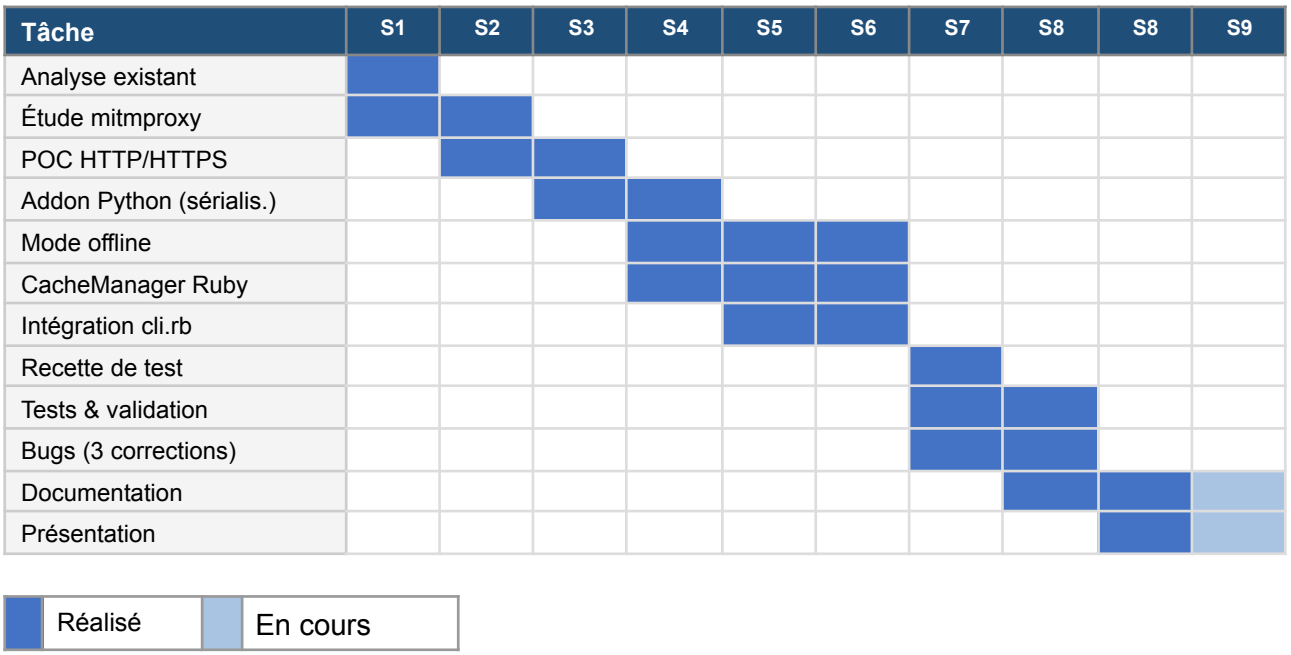
8.1 WBS — Décomposition du travail

ID	Tâche	Livrable
1	Analyse de l'existant (Kameleon, Polipo, engine.rb, cli.rb)	Connaissance de l'architecture interne
1.1	Étude du code source Kameleon (cli.rb, engine.rb, persistent_cache.rb)	Notes d'analyse
1.2	Identification des limitations de Polipo	Tableau comparatif
1.3	Évaluation de mitmproxy comme remplacement	POC standalone validé
2	Développement de l'addon Python	mitmproxy_cache.py
2.1	POC HTTP/HTTPS MISS/HIT	Test curl/wget validé

ID	Tâche	Livrable
2.2	Sérialisation JSON+base64 (API mitmproxy 11.x)	Correction to_bytes()
2.3	Mode offline, hooks error/http_connect/tls_failed	3 scénarios offline validés
3	Intégration Ruby dans Kameleon	cache_manager.rb + cli.rb
3.1	Développement de CacheManager.rb	Démarrage/arrêt mitmdump, archive
3.2	Patch cli.rb (neutralisation polipo, was_enable_cache)	Build fonctionnel
3.3	Déploiement dans le gem installé	Tests kameleon CLI
4	Tests et validation	Recette test_cache.yaml
4.1	Recette de test 10 étapes	Dry-run OK, build OK
4.2	Validation --enable_cache / --from_cache	Archive + replay validés
4.3	Validation réseau coupé (iptables)	Exit 0 sans réseau
5	Documentation et rapport	Ce document

8.2 Planning — Diagramme de Gantt

Période : 19 janvier 2025 (S1) au 9 mars 2025 (S8), soit 10 semaines de travail effectif.



8.3 Suivi du temps et analyse des dépassements

Lot	Estimé	Réel	Écart	Cause du dépassement
1 — Analyse	2 j	2,5 j	+0,5 j	Lecture approfondie engine.rb / persistent_cache.rb
2 — Addon Python	4 j	4 j	0	Conforme aux estimations
3 — Intégration Ruby	5 j	6 j	+1 j	3 bugs bloquants imprévus (voir ci-dessous)
4 — Tests	3 j	4 j	+1 j	Debug mode offline (connexion TCP / connection_strategy)

Lot	Estimé	Réel	Écart	Cause du dépassement
5 — Documentation	2 j	1,5 j	-0,5 j	En cours
TOTAL	16 j	18 j	+2 j	Dépassement de 12,5 %

Analyse des 3 bugs bloquants

Bug	Symptôme	Cause	Solution
1 — API mitmproxy 11.x	AttributeError: to_bytes()	to_bytes() supprimé dans mitmproxy 10+	Sérialisation JSON+base64 maison
2 — Chemin du gem	LoadError: cannot load cache_manager	Kameleon utilise /var/lib/gems/, pas ~/kameleon/	Copie systématique dans le gem + ENV fallback
3 — connection TCP avant hook	MISS malgré cache rempli, serveur éteint	mitmproxy tente TCP avant hook request()	--set connection_strategy=lazy --set upstream_cert=false

8.4 Enjeux éthiques

8.4.1 La technique MITM : entre performance et interception

La technique Man-In-The-Middle (MITM) utilisée par mitmproxy est, par nature, une technique d'interception du trafic chiffré. Bien que son usage soit ici strictement limité à l'optimisation des builds système, son déploiement soulève des questions éthiques légitimes.

Le proxy installe un certificat racine (CA) sur la machine hôte et génère dynamiquement des certificats TLS pour chaque domaine contacté. De l'extérieur, cette opération est identique à une attaque MITM malveillante. La différence fondamentale est le consentement et le périmètre : le proxy est local (127.0.0.1:8080), limité à la durée du build, et installé par l'utilisateur lui-même.

8.4.2 Données stockées dans le cache

Les réponses HTTP/HTTPS sont stockées en clair dans les fichiers JSON du cache. Si des requêtes authentifiées (tokens OAuth, clés API dans les headers) transitent par le proxy, leurs réponses pourraient contenir des informations sensibles.

Recommandations de sécurité :

1. Utiliser KAMELEON_NO_CACHE pour exclure les URLs contenant des tokens
2. Ne jamais versionner les archives .tar.gz dans un dépôt public
3. Supprimer le répertoire de cache après utilisation en environnement partagé
4. Ne pas activer le cache en production sans audit préalable
5. Informer explicitement l'utilisateur de l'installation du certificat CA

8.4.3 Numérique responsable

Sur le plan environnemental, le système contribue à réduire la consommation de bande passante et d'énergie : les paquets déjà mis en cache ne génèrent plus de requêtes vers les serveurs distants, réduisant ainsi le trafic réseau et le temps de calcul des machines de build. C'est un exemple concret d'application du principe de sobriété numérique dans les pratiques DevOps.

9. Conclusion et perspectives

9.1 Bilan

Ce projet a atteint l'objectif principal fixé par le laboratoire IMAG : remplacer Polipo par une solution moderne capable de mettre en cache aussi bien le trafic HTTP que HTTPS, et permettant de rejouer un build intégralement depuis une archive de cache sans connexion réseau. Les trois scénarios de validation — build normal, replay depuis cache, et replay avec réseau coupé — ont tous été validés avec un Exit 0.

La contribution technique principale est l'identification des deux options mitmproxy (connection_strategy=lazy et upstream_cert=false) qui résolvent le problème fondamental de la connexion TCP préalable au hook Python pour le HTTPS. Cette solution, inspirée de l'analyse du projet cproxy et de la documentation mitmproxy, est élégante car elle ne nécessite aucune modification du système d'exploitation (pas de iptables, pas de DNS) et fonctionne sans privilèges root.

9.2 Travaux restants

1. Tester avec une vraie recette Kameleon (debootstrap, apt, pip) pour valider le cache des paquets .deb et .whl en HTTPS
2. Corriger le faux message 'Failed to create archive' (faux positif lié à $CHILD_STATUS nil)
3. Propager http_proxy dans exec_in pour les contextes chroot, qemu et lxc

10.3 Perspectives

À plus long terme, plusieurs améliorations sont envisageables : la mise en place d'un cache partagé entre plusieurs machines via un serveur centralisé, l'invalidation sélective du cache par domaine, ou l'intégration avec les systèmes de CI/CD du laboratoire pour une réutilisation automatique des archives de cache entre pipelines.

Bibliographie :

Documentation officielle

[1] mitmproxy Project. *mitmproxy documentation — version 11.0.2*. 2024. Disponible sur : https://docs.mitmproxy.org/stable/

[2] mitmproxy Project. *Addon development — hooks API*. 2024. Disponible sur : https://docs.mitmproxy.org/stable/addons-overview/

[3] mitmproxy Project. *Options reference — upstream_cert, connection_strategy*. 2024. Disponible sur : https://docs.mitmproxy.org/stable/concepts-options/

[4] Kameleon Project. *Kameleon documentation — recipes and steps*. Laboratoire IMAG, UGA. Disponible sur : https://kameleon.imag.fr/

[5] Kameleon Project. *Kameleon source code — version 2.11.1*. GitHub. Disponible sur : https://github.com/oar-team/kameleon-builder

[6] Python Software Foundation. *hashlib — Secure hashes and message digests*. Python 3.10 documentation. Disponible sur : https://docs.python.org/3/library/hashlib.html

[7] Python Software Foundation. *base64 — Base16, Base32, Base64, Base85 data encodings*. Python 3.10 documentation. Disponible sur : https://docs.python.org/3/library/base64.html

Outils et logiciels utilisés

[8] mitmproxy contributors. *mitmproxy 11.0.2* [logiciel]. 2024. Licence MIT. Disponible sur : https://github.com/mitmproxy/mitmproxy

[9] Kameleon contributors. *kameleon-builder 2.11.1* [gem Ruby]. Laboratoire IMAG. Disponible sur : https://rubygems.org/gems/kameleon-builder

[10] Canonical Ltd. *Ubuntu 22.04 LTS (Jammy Jellyfish)* [système d'exploitation]. 2022. Disponible sur : https://ubuntu.com/

[11] The OpenSSL Project. *OpenSSL 3.4.1* [bibliothèque cryptographique]. 2025. Disponible sur : https://www.openssl.org/

[12] coiby. *cproxy — cache proxy based on mitmproxy* [logiciel]. GitHub. Consulté en mars 2025. Disponible sur : https://github.com/coiby/cproxy

Références techniques

[13] Fielding, R. et al. *RFC 7230 — Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing*. IETF, 2014. Disponible sur : https://datatracker.ietf.org/doc/html/rfc7230

[14] Rescorla, E. *RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3*. IETF, 2018. Disponible sur : https://datatracker.ietf.org/doc/html/rfc8446

[15] Luotonen, A. *Tunneling TCP based protocols through Web proxy servers*. IETF Draft, 1998. Disponible sur : https://datatracker.ietf.org/doc/html/draft-luotonen-web-proxy-tunneling

[16] Belshe, M., Peon, R., Thomson, M. *RFC 7540 — Hypertext Transfer Protocol Version 2 (HTTP/2)*. IETF, 2015. Disponible sur : https://datatracker.ietf.org/doc/html/rfc7540

Pages de référence consultées

[17] Stack Overflow. *mitmproxy — how to cache responses*. Consulté en janvier 2025. Disponible sur : https://stackoverflow.com/questions/tagged/mitmproxy

[18] mitmproxy GitHub Issues. *connection_strategy option — lazy vs eager*. Consulté en mars 2025. Disponible sur : https://github.com/mitmproxy/mitmproxy/issues

[19] Polipo Project. *Polipo — a caching web proxy* [logiciel, abandonné]. Consulté pour analyse comparative. Disponible sur : http://www.pps.univ-paris-diderot.fr/~jch/software/polipo/